

Viva Q&A Preparation

AI-Based Tomato Ripening Stage Detection and Sorting System with Shelf-Life Estimation for the Padma Variety in Sri Lanka

IIT Group 16 · Uva Wellassa University · IIT 474-6

Key number	Value	Key number	Value
Test mAP50 (overall)	0.936	Images / tomatoes	9,600+ / ~11,400
Test mAP50-95	0.711	Test set	777 images
Precision / recall	0.906 / 0.922	Confidence threshold	0.45
Defect mAP50 (test)	0.804	Exit grace / cooldown	0.6 s / 2 s
Validation mAP50	~0.96	Inference (GPU)	6.7 ms

Use the **test-set** figures (0.936) as the official result. The ~0.96 figure is from the validation split. Shaded notes mark things you should prepare with your own measured numbers.

A. Problem, motivation and why Padma

Q1. What problem are you solving?

Manual tomato grading is slow, labour-heavy and subjective. Batches end up mixed, so shelf life is unpredictable and post-harvest losses reach about 20% (FAO 2022). No automated system exists that is tuned for Padma and detects multiple ripening stages plus defects in real time.

Q2. Why Padma?

It is the dominant local cultivar in Badulla and Nuwara Eliya, preferred for high yield and resistance to bacterial wilt. It is climacteric and perishable, and Abekoon et al. (2024) already provide a local baseline for its ripening stages.

Q3. Why is Padma-specific work needed? A tomato is a tomato.

Colour progression, size and skin texture vary by cultivar. A model trained on generic or foreign datasets learns different colour boundaries between stages. Shelf-life timing is also cultivar- and climate-specific.

Q4. What is your final goal in one sentence?

A real-time AI system that detects, classifies and sorts Padma tomatoes by ripeness stage and visible defects, and estimates their remaining shelf life.

Q5. Who benefits?

Farmers, collectors and wholesalers get consistent batches and less waste. Retailers can plan storage and sale times from the shelf-life estimate.

B. Literature and novelty

Q6. What is novel? Abekoon et al. already studied Padma.

Abekoon's work is **offline** image processing on static single-tomato photos, with no object detector and no sorting. Moya et al. (2025) sorted on a conveyor, but with only **3 classes** and no shelf life or KPIs. Geetha et al. (2025) used generic datasets and **constant-speed, timer-based actuation**.

Ours is the **integrated** system:

- real-time YOLOv8 detection of 4 stages plus a defect class, on a local Padma dataset
- physical conveyor sorting triggered by IR sensors
- shelf-life mapping from our own 31-day study
- a live KPI dashboard

Don't claim to be the first to study Padma. Claim the integration and real-time sorting.

Q7. Abekoon used 7 stages. Why only 4 plus defect?

Stages that are close together (pink and light-red, for example) are hard to separate visually and aren't useful for sorting decisions. Four stages map to four physical gates and to meaningful market decisions: store, ship, sell soon, sell now. Defect is a separate reject class.

Q8. How does your work compare with Moya et al.?

Same model family (YOLOv8) and a conveyor as well. We add two more ripeness classes, an explicit defect class, a cultivar-specific dataset, shelf-life estimation, KPI monitoring and IR-triggered gates instead of a fixed timing.

C. Dataset

Q9. How was data collected?

At CLD Farm Fresh, Bandarawela, using a controlled white box with LED lighting. We collected more than 9,600 images in 5 classes and annotated them in Roboflow with bounding boxes.

Q10. How many images, and how many instances?

More than 9,600 images and about 11,400 annotated tomatoes, because some images contain several tomatoes.

Q11. How did you split the data?

Train, validation and test splits from Roboflow. The **test set (777 images) was never used for training or model selection**. Our training script audits for duplicate images across splits, so no data leaks.

Prepare: know your exact train and validation counts.

Q12. How do you define each class?

Class	Definition
Green	100% green, firm
Breaker	Green-yellow, less than 30% red
Turning	30–80% red
Red	More than 80% red, softer
Defect	Cracks, bruising or rot, whatever the colour

Q13. Who labelled the data, and how do you ensure consistency?

The team labelled in Roboflow using the colour-coverage rules above, with cross-checking.

We found and fixed a real consistency bug. Some defect images had a box around the **blemish only**, while others had a box around the **whole tomato**. That broke YOLO's scale assumptions. Re-annotating with whole-tomato boxes raised defect mAP50 from 0.595 to 0.940. The training script now automatically flags classes whose average box size is abnormal.

Q14. Were the classes balanced?

After rebalancing, yes: roughly 2,100–2,450 instances per class. Defect was under-represented after we removed bad images. We generated bounding-box-aware augmented copies (flip, rotate, scale, brightness) **only in the training split**. Validation and test were left untouched, so the results stay honest.

Q15. Why was defect a problem at first?

There were two causes:

- **Annotation inconsistency** (see the labelling question above).
- **Domain confound**. Defect images came from Kaggle, with different backgrounds and lighting from our lightbox photos. The model learned “different lighting means defect” and flagged healthy tomatoes as defect under real lighting. We replaced them with self-captured defect images.

Q16. Doesn't colour augmentation change the ripeness class?

Yes, that's a real risk. Hue jitter was kept small, and the heavy offline augmentation was applied only to the defect class, where colour isn't the defining feature.

D. Model

Q17. Why YOLOv8?

- One pass does both **localisation and classification**, and handles multiple tomatoes in a frame.
- It is fast enough for real time.
- It has mature tooling (Ultralytics, PyTorch).
- It is proven in the most relevant study (Moya et al.).

Q18. Why a single model instead of a detector plus a separate classifier?

One pass means lower latency, a simpler pipeline and nothing to go out of sync between two models. Five classes are few enough that one detection head handles them well.

Q19. Why the medium (m) size and not nano or large?

It is a balance of accuracy and speed. Nano loses accuracy on fine colour and defect detail. Large is slower, with little gain on a 5-class problem. YOLOv8m has about 25.9 million parameters.

Q20. Why a lightbox, when real conditions vary?

Colour is the main feature, so uncontrolled lighting would add noise and confounds (see the defect Kaggle issue). The conveyor itself is an enclosed, lit station, so the lightbox matches deployment. The limitation is generalisation to field lighting, which is future work.

Q21. Briefly explain YOLOv8's architecture.

- **Backbone**: CSPDarknet with C2f blocks, which extract features.
- **Neck**: PAN-FPN, which fuses features at multiple scales.
- **Head**: anchor-free and decoupled, with separate branches for boxes and classes.
- **Loss**: CloU for boxes, binary cross-entropy (BCE) for classes, and distribution focal loss (DFL) to refine box edges.

Q22. Training settings?

- Pretrained YOLOv8m, 640×640 input, up to 150 epochs.
- Early stopping with patience 25. The best checkpoint was epoch 52, and training stopped at epoch 77.
- Initial learning rate $lr_0 = 0.01$ with warmup, automatic batch size, mixed precision (AMP).
- Built-in augmentation: mosaic, horizontal flip, $\pm 15^\circ$ rotation.
- Trained locally on an RTX 3070 Ti (the original baseline was trained on Kaggle).

Q23. What is transfer learning here?

We start from COCO-pretrained weights, so general features such as edges, shapes and textures are already learned. We then fine-tune on our tomato dataset, which needs far less data and time.

Q24. How do you know it isn't overfitting?

Validation loss falls and then levels off without rising again. Early stopping keeps the best validation checkpoint. The held-out test score (0.936) is close to validation (0.96).

E. Metrics and results

Q25. Define precision, recall, F1 and IoU.

- **Precision** = $TP / (TP + FP)$: of everything predicted as a class, how much was correct.
- **Recall** = $TP / (TP + FN)$: of all real objects, how many were found.
- **F1** = $2PR / (P + R)$.
- **IoU** = the overlap between the predicted and true box, divided by their union.

Q26. What are mAP50 and mAP50-95?

- **AP** is the area under the precision–recall curve for one class. **mAP** is the mean over the 5 classes.
- **mAP50** counts a detection as correct when $IoU \geq 0.5$, so it answers “did it find and classify the tomato?”
- **mAP50-95** averages over IoU thresholds from 0.5 to 0.95, so it is stricter and also measures how tight the box is.

Q27. Your final numbers?

Held-out test set (777 images):

Class	mAP50
Overall	0.936 (P 0.906, R 0.922, mAP50-95 0.711)
Green	0.982
Red	0.988
Breaker	0.966
Turning	0.941
Defect	0.804

Q28. Why is mAP50-95 (0.71) much lower than mAP50?

It demands very precise box edges. For sorting we only need correct class and approximate location, so mAP50 matters more for our task.

Q29. Which classes get confused, and why?

Neighbouring ripeness stages: breaker with green and turning, and turning with red. Ripening is a **continuous colour gradient**, so the boundaries between stages are fuzzy even for humans. This is an honest limitation.

Q30. Why is defect the weakest class?

Defects are small, varied (cracks, bruises, rot) and appear on any colour. The class also has the fewest real samples. Our test defect set was also replaced with harder, self-captured images, so 0.804 reflects realistic conditions.

Q31. How do you protect against missed defects?

With a **defect-priority override** in software. If defect appears with confidence of at least 0.80, in at least 2 frames and in at least 35% of a tomato's frames, the tomato is marked defect regardless of any ripeness majority. The fraction rule stops a couple of flickering false-defect frames from rejecting a good tomato.

Q32. Why a confidence threshold of 0.45?

The F1–confidence curve peaks around 0.49–0.53, which is the best balance of precision and recall. 0.45 sits slightly lower so real tomatoes aren't missed, and the multi-frame vote filters out the noise.

Q33. What does 6.7 ms per frame mean?

Model inference time on our GPU. End-to-end time is longer because it includes camera capture and the non-tomato filter model. The decision also waits deliberately for the tomato to leave the view (0.6 s).

Q34. Did you compare other models?

Yes, we compared YOLO26m on the same test set:

- mAP50 is essentially tied (0.930 vs 0.936).
- mAP50-95 is better with YOLO26m (0.785 vs 0.711), meaning tighter boxes.
- Defect is **worse** with YOLO26m (0.754 vs 0.804).

We kept YOLOv8m because defect detection matters most for quality control.

Q35. Slide 22 says 0.96 but slide 26 says 0.936. Which is right?

Both are right, on different splits. 0.96 is the **validation** split used during training. 0.936 is the **held-out test** split the model never saw, and it is the honest, reportable figure.

F. Software pipeline

Q36. Walk through the pipeline.

1. The camera captures a frame.
2. YOLOv8 runs, then the non-tomato filter drops unwanted boxes.
3. The tracking state machine collects votes while the tomato is in view.
4. Once the tomato has been gone for 0.6 s, the class is finalised.
5. One row is saved to SQLite and the dashboard updates.
6. A Bluetooth command (class1–class4, or no class for defect) goes to the ESP32.
7. The ESP32 adds it to a FIFO queue.
8. The IR sensor triggers, and the matching gate opens.

Q37. Why vote across frames instead of using one frame?

A single frame can be wrong because of blur, glare or angle. A **confidence-weighted vote** across all frames (the class with the highest summed confidence wins) is much more stable.

Q38. How do you avoid counting one tomato twice?

Two measures:

- A **0.6 s exit grace**: the tomato must be missing for 0.6 s before it counts as gone.
- A **2 s re-acquire cooldown** after each decision.

Double counting did happen during development: a hand or a brief flicker created a phantom second tomato and shifted the ESP32 queue. These two settings fixed it.

Q39. What happens if a non-tomato object (a phone, a bottle cap) appears?

The model only knows the 5 tomato classes, so it would force the object into one of them. As a stopgap, a second pretrained COCO model (YOLO26n) runs in parallel, and any tomato box that overlaps a detected bottle, phone, hand and so on ($\text{IoU} \geq 0.3$) is dropped. The proper fix is to retrain with background/negative images. That's on the recommendations slide.

Q40. Is this multi-object tracking?

No. The design assumes tomatoes arrive **single-file**, so a simple IDLE/TRACKING state machine is enough. Multi-object tracking (for example ByteTrack) for higher throughput is future work.

Q41. What does the dashboard show?

Built with Streamlit, it shows:

- live camera with boxes and tracking state
- total count and throughput per minute
- class distribution and defect ratio
- average confidence and average shelf life
- recent detections

It also shows the Bluetooth status and a “Reset ESP32 queue” button.

Q42. Does the dashboard show accuracy?

Not live, because there is no ground truth on the belt. It shows **average confidence**. Accuracy is measured offline on the test set.

Q43. Why SQLite?

It is lightweight, needs no server and is enough for single-station logging. Each tomato is logged **once**, not per frame, so the KPIs aren't inflated.

G. Hardware and integration

Q44. What hardware did you use?

- ESP32 (classic, for Bluetooth SPP)
- NEMA stepper with a DM542 driver, and a potentiometer for belt speed
- 4 MG995 servo gates
- 4 IR sensors
- USB camera on an overhead mount
- Laptop for inference

Q45. Why an ESP32?

It is cheap, has built-in Bluetooth Classic and enough I/O for 4 servos, 4 IR sensors and a stepper, and can be programmed with the Arduino framework through PlatformIO.

Q46. Why Bluetooth instead of USB or Wi-Fi?

Bluetooth SPP appears on Windows as a normal COM port, so the code stays simple, and it needs no network setup or cable. The downside is that a line can occasionally be dropped or duplicated, so the firmware supports a sequence number with ACK.

Q47. How does the ESP32 know when to open a gate?

It uses **IR sensors, not timers**:

1. The class is pushed onto stage 0's queue.
2. When the IR sensor at a stage detects a tomato arriving, the ESP32 pops the queue.
3. If the class matches that stage, it opens that gate after a short delay (about 200 ms).
4. If not, the class is passed on to the next stage's queue.

Defect (noclass) never matches, so the tomato rides off the end of the belt.

Q48. Why IR sensors instead of computing time from belt speed?

Belt speed changes (it's adjustable with a potentiometer) and belts slip. An event trigger responds to where the tomato **physically** is. Geetha et al.'s constant-speed timing was a limitation we addressed.

Q49. Why doesn't defect get a gate?

It needs fewer actuators and it is **fail-safe**: anything not positively matched to a ripeness gate ends up in the reject bin at the end of the belt.

Q50. What if a command is lost or duplicated?

The ESP32 queue has no tomato identity, only order, so one extra or missing command shifts every gate after it. Protection comes from:

- the exit grace and cooldown on the Python side
- sequence-number ACKs in the firmware
- a reset command to clear the queue

Q51. What limits belt speed?

The class must reach the ESP32 **before** the tomato reaches the first IR sensor. The 0.6 s exit wait and the Bluetooth time use up that margin. Possible fixes:

- more distance between the camera and the first sensor
- deciding after N consistent frames instead of waiting for the tomato to leave
- a faster model

Q52. What is the throughput?

It is limited mainly by the 2 s cooldown plus the 0.6 s exit wait, about one tomato every 3 s, or roughly 20 per minute. That suits hand-fed, single-file operation. Multi-object tracking is the path to higher throughput.

Prepare: quote your measured figure if you have one.

Q53. How did you test? What were the 4 testing phases?

1. Model evaluation on the held-out test set.
2. Software test in simulated mode, with no hardware, using a webcam.
3. Hardware bench tests of belt, servos, IR sensors and Bluetooth.
4. Full lab run of the complete system with real tomatoes.

Prepare: match these to your slide 14 wording.

Q54. What is your sorting success rate?

Give your measured number: correctly sorted out of the total tried. If you don't have one, say that end-to-end trials are ongoing and don't claim "zero errors".

H. Shelf life

Q55. How was shelf life estimated?

A 31-day observation study at room temperature (22–24 °C) with daily morning checks. For each stage we recorded the days until the tomato became unmarketable:

Stage	Shelf life
Green	31 days
Breaker	29 days
Turning	24 days
Red	12 days
Defect	0 days

The system looks up this value from the detected stage.

Q56. So it's not really prediction?

Correct. It is a **stage-based estimate**, not a per-fruit prediction. Per-fruit dynamic shelf-life estimation is listed as future work.

Q57. How did you decide a tomato's shelf life had ended?

Visible spoilage: softening, wrinkling, leaking, mould or rot.

Prepare: state your exact criterion.

Q58. What are the study's limitations?

One temperature range, no humidity control, only one variety, and a limited sample. Cold storage would extend all of these values.

Prepare: know how many tomatoes were in each stage.

Q59. Why room temperature?

It reflects typical Sri Lankan retail and storage conditions, where there is no cold chain.

I. Limitations, future work and publication

Q60. Main limitations?

- Neighbouring ripeness stages get confused.
- The system depends on laptop inference.
- It assumes single-file flow.
- Shelf life is looked up by stage.
- It has only been tested under lightbox lighting.
- The model has no background or negative class.

Q61. What's next?

From the recommendations slide:

- a stand-alone unit on a Raspberry Pi or Jetson, with no laptop
- a dedicated model to reject non-tomato objects
- running at maximum belt speed
- multi-object tracking for higher throughput
- per-fruit shelf-life estimation

Q62. Where will you publish?

Rolling-submission journals: *Smart Agricultural Technology* (where Moya and Abekoon published), *Computers and Electronics in Agriculture*, or *MDPI Sensors/Agriculture*.

Q63. Cost compared with commercial sorters?

Commercial optical sorters cost thousands to tens of thousands of dollars. Ours is built from low-cost parts: ESP32, hobby servos and a USB camera.

Prepare: know your parts cost.

J. Questions for Gayan (model and integration lead)

Q64. Why did you retrain the model several times?

- The Kaggle baseline had a weak defect class (0.595).
- Re-annotation raised it to 0.940, but live testing showed healthy tomatoes flagged as defect because of the domain confound.
- The version 6 retrain used self-captured defect images and balancing, and it is the model deployed now.

Q65. What was the hardest integration bug?

Duplicate commands for one tomato shifting the ESP32 FIFO queue, so every later tomato went to the wrong gate. It was fixed with a time-based exit grace and the 2 s cooldown, confirmed against the ESP32 queue logs.

Q66. Why did you change the exit rule from frame-based to time-based?

Inference speed varies with system load. A fixed number of frames lasts a different amount of real time on each run, so seconds are more reliable.